OOPS PROJECT • TECHNICAL VIVA REFERENCE

College Admission Portal

State CET Cell Centralized Admission Process (CAP) • Architecture, Code & Viva Questions Guide

1. Object-Oriented Programming (OOP) Concepts in the Project

Q1. How is Abstraction implemented in this project?

Abstraction is implemented via the `abstract class User` in `User.java`.

- `User` defines the essential structure and state of an authenticated user (`id`, `name`, `email`, `password`, `phoneNumber`, `role`), but cannot be instantiated directly with `new User(...)`.
- It defines the abstract method `public abstract String getRoleDescription();` which hides internal implementation details while requiring concrete subclasses (`Student`, `Admin`) to provide their own role explanations.

Q2. Where is Inheritance used and what benefits does it provide?

Inheritance is demonstrated by `Student extends User` and `Admin extends User`.

- **Code Reusability:** Common identity fields and getter/setter methods are written once in `User` and automatically inherited, avoiding duplication.
- **Constructor Chaining:** Both subclasses call `super(id, name, email, password, phoneNumber, "STUDENT" / "ADMIN")` to initialize inherited fields in the parent class.

Q3. How does the project demonstrate Polymorphism (both compile-time and run-time)?

- **Run-time Polymorphism (Method Overriding):** `User` declares `public abstract String getRoleDescription();`. `Student` returns student applicant details, while `Admin` returns administrator permissions. When `displayUserDetails()` executes, the method body dispatched is determined dynamically at runtime based on the underlying object.
- **Compile-time Polymorphism (Method Overloading):**
  - `Application.java` features multiple overloaded constructors (Default, 10-field backward-compatible, and 16-field extended constructors).
  - `AdmissionSystem.java` overloads `submitApplication()` to accept either a single course choice or a prioritized `List<String> branchPreferences`.

Q4. How is Encapsulation enforced across the backend classes?

Encapsulation (data hiding) is enforced across `User`, `Student`, `Admin`, `Application`, `Document`, and `AdmissionSystem`:

- All member variables and internal ArrayList collections are declared `private`.
- Controlled read and write access is exposed exclusively through `public` getters and setters.
- Direct data corruption is prevented, and collection mutations are governed by verified controller methods.

2. Backend Architecture & Web Server Networking

Q5. How does the web server work without external frameworks like Spring Boot or Tomcat?

The project uses Core Java Standard Library's built-in HTTP server: `com.sun.net.httpserver.HttpServer`.

- `HttpServer.create(new InetSocketAddress(8080), 0)` binds the server to port 8080 in `Main.java`.
- HTTP endpoints are mapped to inner classes implementing `HttpHandler` using `server.createContext(path, handler)`.
- The handler reads request streams, interacts with `AdmissionSystem`, and sends HTTP response headers and JSON/HTML bytes via `HttpExchange`.

Q6. How are static files (HTML, CSS, JS, Images) served to the client browser?

Static files are served through `StaticFileHandler` registered at route `/`:

- `locateFrontendFolder()` searches candidate locations (`frontend`, `CollegeAdmissionPortal/frontend`, `../frontend`) to reliably resolve the absolute path.
- Requests for `/` or empty paths resolve to `index.html`.
- Clean routes without `.html` extensions (e.g. `/status`, `/register`) automatically fallback to their `.html` files.
- Appropriate MIME types (`text/html`, `text/css`, `image/jpeg`, `application/pdf`) are transmitted along with the byte stream.

Q7. How does the server handle concurrency and thread-safety during file updates?

Critical write and state update methods in `AdmissionSystem.java` (e.g. `registerStudent()`, `submitApplication()`, `saveApplicationsToFile()`, `allocateBranch()`) are declared with the `synchronized` keyword. This provides a mutual-exclusion monitor lock ensuring multiple concurrent HTTP worker threads cannot interleave file writes or corrupt in-memory ArrayLists.

3. File Handling, Storage & Serialization

Q8. How does the persistence layer work without an SQL database?

Data is persisted in pipe-delimited flat files in the `data/` folder:

- `students.txt` : `StudentID|FullName|Email|Password|PhoneNumber`
- `admin.txt` : `AdminID|FullName|Email|Password|PhoneNumber`
- `applications.txt` : 16 serialized fields including branch preferences (semicolon-separated) and uploaded document pairs (`docType:fileName;...`).
- On startup, `BufferedReader` reads each record and parses it using `fromDataString()` . When updated, `BufferedWriter` flushes serialized `toDataString()` lines back to disk.

Q9. How are uploaded candidate documents stored and served?

Candidate uploads are submitted via `POST /api/upload-document` :

- Payload size is validated against a 5 MB ceiling, and extensions are restricted to `.pdf` , `.jpg` , `.jpeg` , and `.png` .
- Files are written to organized server-side directories: `data/documents/<appId>/<docType>_<fileName>` .
- `DocumentViewHandler` serves documents with `Content-Disposition: inline` for in-browser preview or `attachment` for direct download.

4. Admission Workflow & Centralized System Lifecycle

Q10. What is the full lifecycle of an Application in the portal?

```
1. Student Registration & Authentication (register.html / login.html) ↓ 2. Academic Scores & 6 Ordered Branch Preferences Entry (application.html) ↓ 3. Upload of Verification Documents: Aadhaar, SSC, HSC, CET Scorecard [Status: "Pending" | Allocation: "Pending Allocation" | Verification: "Pending Verification"] ↓ 4. Admin Scrutiny & Verification of Uploaded Credentials (admin-dashboard.html) [Status: "Documents Verified"] ↓ 5. Admin Merit-based Branch Allocation with remarks [Status: "Branch Allocated" | Allocation Status: "Awaiting Student Acceptance"] ↓ 6. Candidate Provisional Allotment Review: ├── ACCEPT ALLOCATION ──→ Status: "Admission Confirmed" (Seat Freezed) └── REJECT ALLOCATION ──→ Status: "Allocation Rejected - Pending Reallocation"
```

5. Quick Reference Viva Trap Questions

Viva Question	Correct Technical Explanation
Why not use JDBC / MySQL?	The core objective is mastering Object-Oriented Design, Java File I/O (`BufferedReader` / `BufferedWriter`), and byte stream manipulation without external drivers or container overhead.
Why is `AdmissionSystem` declared `final`?	Calling overridable methods (like `loadAdminsFromFile()`) from within a constructor triggers a `[this-escape]` warning in modern Java. Declaring the class `final` ensures it cannot be subclassed, eliminating this risk.
What does `split("\\|", -1)` do?	`\\|` escapes the pipe delimiter in regex. The limit `-1` preserves trailing empty tokens so optional empty fields (e.g. remarks or allocation status) are not dropped.
Why does `start.bat` check port 8080 with `netstat`?	If a prior JVM process terminated unexpectedly, the port remains bound, causing a fatal `BindException: Address already in use`. The script automatically identifies the PID and frees the port.